Computability – Topic 9

Yonghao Lee

March 2, 2026

Contents

1 More on P and NP **2**
1.1 Closure Properties of P and NP 2
1.2 The Class coNP 3
1.3 P Versus NP 4

2 NP-Completeness **4**
2.1 The Satisfiability Problem 5

3 The Cook–Levin Theorem **8**

1 More on P and NP

Recall that we have defined the two complexity classes:

$$P = \bigcup_{k=0}^{\infty} \text{TIME}(n^k)$$

and

$$\text{NP} = \bigcup_{k=0}^{\infty} \text{NTIME}(n^k)$$

That is, NP is the set of languages that can be decided by a Nondeterministic Turing Machine (NTM) in polynomial time.

The second, equivalent definition of NP relies on the concept of a verifier:

Alternative Definition of NP: A language L is in NP if there exists a deterministic polynomial-time Turing machine V (a verifier) and a polynomial p such that:

$$L = \{w \mid \exists c \text{ with } |c| \leq p(|w|) \text{ such that } V \text{ accepts } \langle w, c \rangle\}$$

Explanation: In plain English, a string w is in the language L if and only if there exists a "certificate" c that proves it. This certificate must be relatively short (its length is bounded by a polynomial p relative to the length of w). The verifier V takes both the input w and the certificate c and can verify the proof in polynomial time.

1.1 Closure Properties of P and NP

We prove some closure properties here.

Theorem 1.1: P is closed under union

If $L_1 \in P$ and $L_2 \in P$, then $L_1 \cup L_2 \in P$.

Proof. Let $L_1, L_2 \in P$, and let M_1, M_2 be the deterministic Turing machines that decide L_1 and L_2 respectively in polynomial time. We define a new TM M that decides $L_1 \cup L_2$ as follows:

$M =$ On input w:

1. Run M_1 on w. If it accepts, *accept*.
2. Run M_2 on w. If it accepts, *accept*.
3. If neither accepted, *reject*.

Complexity Analysis: The runtime of M is simply the sum of the runtimes of M_1 and M_2 (plus a small constant overhead for switching machines). Since M_1 and M_2 both run in polynomial time, and the sum of two polynomials is always a polynomial, M runs in polynomial time. The language decided by M is precisely $L_1 \cup L_2$, proving that P is closed under union. □

Theorem 1.2: P is closed under complementation

If $L \in P$, then $\overline{L} \in P$.

Proof. Let $L \in P$ and let M be a deterministic TM that decides L in polynomial time. We construct a new TM M' that decides the complement language $\overline{L}$.

To build M', we simply take the machine M and swap its accepting and rejecting states. Because M is a decider (it always halts), M' will also always halt. For any input w, if M accepts, M' rejects; if M rejects, M' accepts.

Since flipping the states does not add any computational steps to the execution, M' runs in the exact same polynomial time as M. Therefore, $\overline{L} \in P$. □

Remark 1.1: Summary of Closure Properties

P is closed under union, intersection, complementation, concatenation, and Kleene star.
NP is closed under union, intersection, and concatenation. Whether NP is closed under complementation is equivalent to the open question of whether NP = coNP.

Now consider the SUBSET-SUM problem concerning integer arithmetic. We are given a collection of numbers $x_1, \ldots, x_k$ and a target number t. We want to determine whether the collection contains a subcollection that adds up to t. Formally:

$$\text{SUBSET-SUM} = \left\{ \langle S, t \rangle \;\middle|\; \begin{array}{l} S = \{x_1, \ldots, x_k\} \text{ is a multiset of integers and } t \text{ is an integer,} \\ \text{such that } \exists\, \{y_1, \ldots, y_j\} \subseteq S \text{ with } \sum_{i=1}^{j} y_i = t \end{array} \right\}$$

Note that S is a multiset, meaning repetition of elements is allowed.

Theorem 1.3: SUBSET-SUM is in NP

The language SUBSET-SUM is in the class NP.

Proof. We prove membership in NP by constructing a polynomial-time verifier, and then give an equivalent NTM.

Method 1: Polynomial-Time Verifier

For a given instance $\langle S, t \rangle$, the certificate c is a subcollection of elements from S that supposedly sums to t.

V = On input $\langle \langle S, t \rangle, c \rangle$:

1. Test whether c is a subcollection of S (i.e., every element in c appears in S with sufficient multiplicity).
2. Compute the sum of the elements in c and test whether it equals t.
3. If both tests pass, *accept*; otherwise, *reject*.

Complexity Analysis: Step 1 takes at most $O(k^2)$ time (comparing elements of c against S). Step 2 takes $O(k)$ additions, each on numbers of polynomial bit-length. Both steps are polynomial in the input size, so V is a polynomial-time verifier. (Note: the input size includes the bit-lengths of all numbers in S, so the additions are polynomial in the total input length.)

Method 2: Nondeterministic Turing Machine

N = On input $\langle S, t \rangle$:

1. Nondeterministically select a subcollection c of the numbers in S.

   (**Explanation:** For each element in S, the NTM nondeterministically chooses whether to include it in c or not. This creates 2^k branches, one for each possible subcollection.)
2. Compute the sum of the elements in c and test whether it equals t.
3. If it equals t, *accept*; otherwise, *reject*.

Complexity Analysis: Step 1 takes $O(k)$ time on each branch (one binary choice per element). Step 2 takes $O(k)$ additions. Since every branch runs in polynomial time, N is a nondeterministic polynomial-time TM. Thus, SUBSET-SUM $\in$ NP. □

1.2 The Class coNP

Sometimes we want to verify that something is *not* a member of a language. This appears to be harder than verifying membership. For example, consider $\overline{\text{HAMPATH}}$: to verify that a graph does *not* have a Hamiltonian path, we would seemingly need to check all $n!$ possible orderings of nodes—we do not know of a short certificate that a verifier could check in polynomial time.

This motivates a separate complexity class:

Definition 1.1: The Class coNP

coNP is the class of languages whose *complements* are in NP. That is,

$$\text{coNP} = \{L \mid \overline{L} \in \text{NP}\}.$$

Equivalently, $L \in \text{coNP}$ if and only if there exists a polynomial-time verifier for $\overline{L}$—i.e., membership in $\overline{L}$ can be efficiently verified given an appropriate certificate.

Remark 1.2: P, NP, and coNP

We know that $P \subseteq \text{NP} \cap \text{coNP}$, because if L is decidable in polynomial time, then both L and $\overline{L}$ have polynomial-time verifiers (the decider itself can serve as the verifier, ignoring the certificate). Whether NP = coNP remains an open question.

1.3 P Versus NP

In simple terms, we can characterize P and NP as follows:

- P = the class of languages for which *membership can be decided* quickly (in polynomial time).
- NP = the class of languages for which *membership can be verified* quickly (in polynomial time, given an appropriate certificate).

Since any polynomial-time decider can serve as a verifier (simply ignore the certificate), we have $P \subseteq \text{NP}$. We have presented examples of languages—such as HAMPATH and CLIQUE—that are in NP but are not known to be in P. The power of polynomial verifiability seems to be much greater than that of polynomial decidability.

However, hard as it may be to imagine, P and NP could be equal; we are currently unable to prove the existence of a single language in NP that is not in P.

Definition 1.2: The Class EXPTIME

EXPTIME is the class of languages decidable in exponential time on a deterministic Turing machine:

$$\text{EXPTIME} = \bigcup_{k \in \mathbb{N}} \text{TIME}(2^{n^k})$$

The best known deterministic algorithms for deciding arbitrary languages in NP use exponential time. In other words, we can prove that

$$P \subseteq \text{NP} \subseteq \text{EXPTIME}.$$

Whether either of these containments is strict remains open, though we do know (by the time hierarchy theorem) that $P \neq \text{EXPTIME}$, so at least one of the two inclusions must be strict.

2 NP-Completeness

One important advance on the P versus NP question came in the early 1970s with the work of Stephen Cook and Leonid Levin. They discovered certain problems in NP whose individual complexity is related to that of the *entire class*. If a polynomial-time algorithm exists for any one of these problems, then *all* problems in NP would be solvable in polynomial time. These problems are called **NP-complete**.

The practical significance of NP-completeness is twofold:

1. **Hardness evidence.** Proving that a problem is NP-complete is strong evidence that it cannot be solved in polynomial time. While we cannot yet prove $P \neq \text{NP}$, the fact that decades of effort have failed to find polynomial-time algorithms for *any* NP-complete problem makes their intractability widely believed.

2. **Avoiding futile searches.** If you can show that your problem is NP-complete, you know not to waste time searching for an exact polynomial-time algorithm. Instead, you can focus on approximation algorithms, heuristics, or special-case solutions.

2.1 The Satisfiability Problem

The first NP-complete problem we present is called the **satisfiability problem**.

Recall that variables that can take on the values TRUE and FALSE are called **Boolean variables**. The Boolean operations AND, OR, and NOT are represented by the symbols $\wedge$, $\vee$, and $\overline{}$, respectively.

A **Boolean formula** is an expression involving Boolean variables and Boolean operations. For example,

$$\phi = (\overline{x} \wedge y) \vee (x \wedge \overline{z})$$

is a Boolean formula. A Boolean formula is **satisfiable** if some assignment of 0s and 1s to the variables makes the formula evaluate to 1.

The example above is satisfiable because the assignment $x = 0$, $y = 1$, $z = 0$ makes ϕ evaluate to 1. We say that the assignment *satisfies* ϕ. The satisfiability problem is to test whether a Boolean formula is satisfiable:

$$\text{SAT} = \{\langle\phi\rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$$

Now we state a theorem that links the complexity of SAT to the complexity of all problems in NP.

Theorem 2.1: SAT and P vs. NP

$\text{SAT} \in P \iff P = \text{NP}$

We defer the proof of this theorem. The ($\Leftarrow$) *direction is immediate since* $\text{SAT} \in \text{NP}$. *The* ($\Rightarrow$) *direction follows from the Cook–Levin theorem (Theorem 3.1), which shows that SAT is NP-complete.*

Now we develop the method that is central to the proof of this theorem.

We defined the concept of reducing one problem to another. When problem A reduces to problem B, a solution to B can be used to solve A. Now we define a version of reducibility that takes the efficiency of computation into account. When A is efficiently reducible to B, an efficient solution to B can be used to solve A efficiently.

Definition 2.1: Polynomial-Time Computable Function

A function $f : \Sigma^* \to \Sigma^*$ is a **polynomial-time computable function** if there exists a polynomial-time deterministic Turing machine M that, when started on any input w, halts with exactly $f(w)$ written on its tape.

Definition 2.2: Polynomial-Time Mapping Reducibility

A language A is **polynomial-time mapping reducible** to a language B, written $A \leq_P B$, if there exists a polynomial-time computable function $f : \Sigma^* \to \Sigma^*$ such that for every $w \in \Sigma^*$,

$$w \in A \iff f(w) \in B.$$

The function f is called a **polynomial-time reduction** of A to B.

As with an ordinary mapping reduction, a polynomial-time reduction of A to B provides a way to convert membership testing in A to membership testing in B, but now the conversion is done efficiently. To test whether $w \in A$, we use the reduction f to map w to $f(w)$ and then test whether $f(w) \in B$.

If one language is polynomial-time reducible to a language already known to have a polynomial-time solution, we obtain a polynomial-time solution to the original language:

Theorem 2.2: Polynomial-Time Reduction Preserves Membership in P

If $A \leq_P B$ and $B \in P$, then $A \in P$.

Proof. Let M be the polynomial-time algorithm deciding B, and let f be the polynomial-time reduction from A to B. We describe a polynomial-time algorithm N deciding A as follows:

$N =$ On input w:

1. Compute $f(w)$.
2. Run M on input $f(w)$ and output whatever M outputs.

Correctness: By the definition of polynomial-time reduction, $w \in A \iff f(w) \in B$. Since M decides B, it accepts $f(w)$ if and only if $f(w) \in B$. Therefore N accepts w if and only if $w \in A$.

Running time: Suppose f runs in time $O(n^a)$ and M runs in time $O(m^b)$, where $n = |w|$ and m is the length of the input to M. Stage 1 computes $f(w)$ in $O(n^a)$ time, so in particular $|f(w)| \leq O(n^a)$ (since the machine cannot write more symbols than the number of steps it takes). Stage 2 runs M on an input of length at most $O(n^a)$, so it takes $O((n^a)^b) = O(n^{ab})$ time. The total running time is $O(n^a + n^{ab}) = O(n^{ab})$, which is polynomial. Hence $A \in P$. □

Now we introduce 3SAT, a special case of the satisfiability problem in which all formulas are in a restricted form.

Definition 2.3: Literals, Clauses, and CNF

A **literal** is a Boolean variable or a negated Boolean variable, such as x or $\overline{x}$.
A **clause** is several literals connected with ORs, such as $(x_1 \vee \overline{x_2} \vee \overline{x_3} \vee x_4)$.
A Boolean formula is in **conjunctive normal form (CNF)** if it is composed of several clauses connected with ANDs, for example:

$$(x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_3 \vee x_4) \wedge (x_2 \vee \overline{x_4})$$

A formula is a **3cnf-formula** if it is in CNF and every clause has exactly three literals, for example:

$$(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4) \wedge (\overline{x_3} \vee x_1 \vee \overline{x_4})$$

We define:

$$\text{3SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$$

Observe that if an assignment satisfies a cnf-formula, then each clause must contain at least one literal that evaluates to 1. This is because the formula is an AND of clauses, so every clause must be true; and each clause is an OR of literals, so at least one literal in it must be true.

The following theorem presents a polynomial-time reduction from the 3SAT problem to the CLIQUE problem.

Theorem 2.3: 3SAT $\leq_P$ CLIQUE

3SAT is polynomial-time reducible to CLIQUE.

Proof. We construct a polynomial-time reduction f that converts 3cnf-formulas into graphs. The key idea is that in the constructed graph, k-cliques correspond exactly to satisfying assignments of the formula. The structure of the graph is designed to mimic the behavior of the variables and clauses.

Construction. Let ϕ be a 3cnf-formula with k clauses:

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$$

The reduction f maps $\langle \phi \rangle$ to $\langle G, k \rangle$, where G is an undirected graph built as follows:

1. **Nodes:** Organize the nodes of G into k groups of three, called **triples** $t_1, t_2, \ldots, t_k$. Triple t_i corresponds to clause i, and its three nodes correspond to the three literals in that clause. Label each node with its corresponding literal.
2. **Edges:** Start by connecting *every* pair of nodes, then remove edges in exactly two cases:
   - **Same-triple rule:** Remove all edges between nodes in the same triple.

     (**Purpose:** This forces any k-clique to pick at most one node from each triple — i.e., at most one literal per clause.)
   - **Contradiction rule:** Remove all edges between two nodes whose labels are contradictory (one is x and the other is $\overline{x}$ for some variable x).

     (**Purpose:** This prevents a clique from selecting both x and $\overline{x}$, which would be an inconsistent assignment.)

Correctness. We show that ϕ is satisfiable if and only if G has a k-clique.

($\Rightarrow$) Satisfying assignment $\implies$ k-clique.

Suppose ϕ has a satisfying assignment. Since every clause must be true under this assignment, at least one literal evaluates to 1 in each clause. From each triple t_i, select one node whose label is a literal that is true under the assignment. (If more than one literal is true in a particular clause, choose one arbitrarily.) This gives us k selected nodes, one from each triple.

We claim these k nodes form a clique. To verify this, we must check that every pair of selected nodes is connected by an edge. A pair could fail to be connected only if it falls under one of the two removal rules:

- **Same-triple rule:** Cannot apply, because we selected exactly one node per triple. Two different selected nodes necessarily come from two different triples.
- **Contradiction rule:** Cannot apply, because all selected literals are simultaneously true under the same assignment. If we had selected a node labeled x and another labeled $\overline{x}$, then the assignment would require both $x = 1$ and $x = 0$, which is impossible.

Since neither removal rule applies to any pair of selected nodes, all pairs are connected. Therefore, the k selected nodes form a k-clique in G.

($\Leftarrow$) k-clique $\implies$ satisfying assignment.

Suppose G has a k-clique S. We show how to extract a satisfying assignment for ϕ from S.

Step 1: S contains exactly one node from each triple. Nodes within the same triple are not connected by edges (same-triple rule), so no two nodes of a clique can come from the same triple. Since S has k nodes and there are k triples, S must contain exactly one node from each triple.

Step 2: The labels on S are consistent. Nodes with contradictory labels are not connected by edges (contradiction rule). Since every pair of nodes in a clique must be connected, no two nodes in S can have contradictory labels. This means we can set all the literals labelling the clique nodes to true without any conflicts.

Step 3: Construct the assignment. Assign truth values to the variables of ϕ so that each literal labelling a clique node is made true. (For any variable that does not appear as a label of a clique node, assign its value arbitrarily.)

Step 4: This assignment satisfies ϕ. Each triple contains exactly one clique node, and that node's literal is true under our assignment. This means each clause contains at least one true literal, so each clause evaluates to 1. Since all clauses are true, ϕ is satisfied.

Running time. The graph G has $3k$ nodes. Constructing the nodes and determining which edges to include requires checking $O(k^2)$ pairs, each in constant time. The entire construction runs in time polynomial in the length of $\langle\phi\rangle$. Therefore, f is a polynomial-time reduction and 3SAT $\leq_P$ CLIQUE. $\square$

The seemingly rather different problems end up being similar in complexity, thanks to polynomial-time reducibility linking their complexities.

Definition 2.4: NP-Completeness

A language B is **NP-complete** if it satisfies two conditions:

1. $B \in \mathrm{NP}$, and
2. Every $A \in \mathrm{NP}$ is polynomial-time reducible to B (i.e., $A \leq_P B$).

Condition 2 alone (without requiring $B \in \mathrm{NP}$) defines what it means for B to be **NP-hard**.

Corollary 2.1: NP-Completeness and P vs. NP

If B is NP-complete and $B \in P$, then $P = \mathrm{NP}$.

Proof. Let A be any language in NP. Since B is NP-complete, $A \leq_P B$. Since $B \in P$, we can decide A in polynomial time by first applying the reduction and then using the polynomial-time algorithm for B. Since A was arbitrary, $\mathrm{NP} \subseteq P$, giving $P = \mathrm{NP}$. □

Theorem 2.4: NP-Completeness is Transferable via Reduction

If B is NP-complete and $B \leq_P C$ for some $C \in \mathrm{NP}$, then C is NP-complete.

Proof. We already know that $C \in \mathrm{NP}$, so we must show that every $A \in \mathrm{NP}$ satisfies $A \leq_P C$.

Since B is NP-complete, every language $A \in \mathrm{NP}$ is polynomial-time reducible to B, i.e., $A \leq_P B$. By assumption, $B \leq_P C$. Polynomial-time reductions compose: if f is a polynomial-time reduction from A to B and g is a polynomial-time reduction from B to C, then $g \circ f$ is a polynomial-time reduction from A to C. (This holds because the composition of two polynomial-time computable functions is polynomial-time computable, as shown earlier.)

Therefore $A \leq_P C$ for every $A \in \mathrm{NP}$, so C is NP-complete. □

3 The Cook–Levin Theorem

Once we have one NP-complete problem, we can obtain others by polynomial-time reduction from it. The Cook–Levin theorem establishes the *first* NP-complete problem.

Theorem 3.1: Cook–Levin Theorem

SAT is NP-complete.

Proof Sketch. We must verify both conditions of NP-completeness.

Condition 1: SAT $\in$ NP. Given a Boolean formula ϕ with variables $x_1, \ldots, x_m$, a certificate is simply an assignment of truth values to the variables. A verifier evaluates ϕ under this assignment in time polynomial in the length of ϕ. If ϕ evaluates to 1, accept; otherwise, reject. Therefore SAT $\in$ NP.

Condition 2: Every $A \in \mathrm{NP}$ satisfies $A \leq_P \mathrm{SAT}$. This is the hard part. Let A be any language in NP, and let N be a nondeterministic polynomial-time TM that decides A in time n^k. We construct a polynomial-time reduction that maps any string w to a Boolean formula ϕ such that:

$$w \in A \iff \phi \text{ is satisfiable}$$

The key idea is to represent the *entire computation* of N on input w as a Boolean formula. We do this using a **tableau**—a table whose rows represent successive configurations of N:

- The tableau is an $n^k \times n^k$ table (since N runs for at most n^k steps and the tape uses at most n^k cells).

- Each cell of the tableau holds a symbol from a finite set $C = Q \cup \Gamma \cup \{\#\}$, where Q is the state set, Γ is the tape alphabet, and $\#$ is a delimiter.
- Row i represents the configuration of N at time step i.

We introduce Boolean variables to encode the contents of each cell, then build ϕ as the AND of four groups of clauses:

1. ϕ_{cell}: ensures every cell in the tableau contains exactly one symbol from C.
2. ϕ_{start}: ensures the first row is the starting configuration of N on input w.
3. ϕ_{move}: ensures each row follows legally from the previous one according to N's transition function. This is checked locally using *windows*—small groups of adjacent cells across two consecutive rows.
4. ϕ_{accept}: ensures that an accepting state appears somewhere in the tableau.

The formula $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$ is satisfiable if and only if there exists a valid accepting computation of N on w—which happens if and only if $w \in A$.

Running time: The tableau has $O(n^{2k})$ cells, each requiring a constant number of Boolean variables. Each group of clauses can be generated in time polynomial in the tableau size. Therefore the entire reduction runs in polynomial time. □

Thus we have concluded the proof of the Cook–Levin theorem, showing that SAT is NP-complete. Showing the NP-completeness of other languages generally does not require such a lengthy proof. Instead, NP-completeness can be proved with a polynomial-time reduction from a language that is already known to be NP-complete. We could use SAT for this purpose, but using 3SAT is usually easier. Therefore, we first show that 3SAT itself is NP-complete.

Theorem 3.2: 3SAT is NP-Complete

3SAT is NP-complete.

Proof. We verify both conditions of NP-completeness.

Condition 1: 3SAT $\in$ NP. Since 3SAT is a special case of SAT (every 3cnf-formula is a Boolean formula), the same polynomial-time verifier works: given a truth assignment as a certificate, evaluate the formula and accept if it evaluates to 1. Therefore 3SAT $\in$ NP.

Condition 2: 3SAT is NP-hard. We show SAT $\leq_P$ 3SAT. Since SAT is NP-complete, every language $A \in$ NP satisfies $A \leq_P$ SAT. By composing with our reduction, we get $A \leq_P$ 3SAT for every $A \in$ NP.

Reduction from SAT to 3SAT. Given an arbitrary Boolean formula ϕ, we convert it into an equivalent 3cnf-formula ϕ' (satisfiable if and only if ϕ is satisfiable) in polynomial time. The conversion proceeds in stages:

1. **Parse into a tree.** Represent ϕ as a binary parse tree, where leaves are variables or their negations, and internal nodes are $\wedge$, $\vee$, or $\neg$ operations.
2. **Introduce auxiliary variables.** For each internal node i of the parse tree, introduce a fresh variable y_i that represents the output of that subexpression. Write a small formula relating y_i to its children. For instance, if node i computes $y_i = y_j \vee y_k$, we write the constraint $y_i \leftrightarrow (y_j \vee y_k)$.
3. **Conjoin all constraints.** The new formula is:

$$\phi' = y_{\text{root}} \wedge \bigwedge_i (y_i \leftrightarrow f_i(y_{\text{children}}))$$

   where y_{root} is the variable for the root node (forcing the overall formula to be true), and each constraint enforces that the auxiliary variable correctly represents its subexpression.
4. **Convert each constraint to 3CNF.** Each constraint $y_i \leftrightarrow f_i(\ldots)$ involves at most 3 variables and can be rewritten as a conjunction of clauses. If any clause has fewer than 3 literals, pad it by

duplicating a literal (e.g., $(a \vee b)$ becomes $(a \vee b \vee b)$). If any clause has more than 3 literals, split it using additional auxiliary variables:

$$(l_1 \vee l_2 \vee l_3 \vee l_4) \;\equiv\; (l_1 \vee l_2 \vee z) \wedge (\overline{z} \vee l_3 \vee l_4)$$

where z is a fresh auxiliary variable. This splitting can be applied repeatedly until all clauses have exactly 3 literals.

The resulting formula ϕ' is in 3CNF, and ϕ' is satisfiable if and only if ϕ is satisfiable. The construction runs in polynomial time because the parse tree has size linear in $|\phi|$, and each step introduces at most a constant number of new variables and clauses per node. Therefore SAT $\leq_P$ 3SAT, and 3SAT is NP-complete. $\square$